Kishore S

CH.SC.U4CSE24222

Week – 6

Design and Analysis of Algorithm(23CSE211)

Quick Sort by taking three different pivots and Quick Select

1. Write a c program for Quick Sort by taking three different pivots

Code:

```
#include <stdio.h>

#include <stdlib.h>

void swap(int* a, int* b) {

    int t = *a;

    *a = *b;

    *b = t;

}

int partition(int arr[], int low, int high, int pivotChoice) {

    int pivotIndex;

    if (pivotChoice == 1) {

        pivotIndex = low;

    } else if (pivotChoice == 2) {

        pivotIndex = high;

    } else {

        pivotIndex = low + rand() % (high - low + 1);

        printf("Random Pivot Element Chosen: %d\n", arr[pivotIndex]);

    }

    swap(&arr[pivotIndex], &arr[high]);

    int pivot = arr[high];

    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {

        if (arr[j] < pivot) {

            i++;

            swap(&arr[i], &arr[j]);

        }

    }

    swap(&arr[i + 1], &arr[high]);
```

```
    return (i + 1);
}
void quickSort(int arr[], int low, int high, int pivotChoice) {
    if (low < high) {
        int pi = partition(arr, low, high, pivotChoice);
        quickSort(arr, low, pi - 1, pivotChoice);
        quickSort(arr, pi + 1, high, pivotChoice);
    }
}
int main() {
    printf("CH.SC.U4CSE24222\n");
    int n, pivotChoice;
    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1) return 1;
    int *arr = (int *)malloc(n * sizeof(int));
    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    printf("Choose Pivot Element:\n1. First Element\n2. Last Element\n3.
Random Element\nChoice: ");
    scanf("%d", &pivotChoice);
    quickSort(arr, 0, n - 1, pivotChoice);
    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    free(arr);
    return 0;
```

}

OUTPUT:

```
kishore@kishore-LOQ-15IRX9: /media/kishore/Kishore Edu/Studies/sem 4/DAA/week 6
kishore@kishore-LOQ-15IRX9:/media/kishore/Kishore Edu/Studies/sem 4/DAA/week 6$ gcc QuickSort.c -o on
e
kishore@kishore-LOQ-15IRX9:/media/kishore/Kishore Edu/Studies/sem 4/DAA/week 6$ ./one
CH.SC.U4CSE24222
Enter number of elements: 4
Enter 4 elements: 55
66
54
56
Choose Pivot Element:
1. First Element
2. Last Element
3. Random Element
Choice: 2
Sorted array: 54 55 56 66
kishore@kishore-LOQ-15IRX9:/media/kishore/Kishore Edu/Studies/sem 4/DAA/week 6$
```

```
kishore@kishore-LOQ-15IRX9: /media/kishore/Kishore Edu/Studies/sem 4/DAA/week 6
kishore@kishore-LOQ-15IRX9:/media/kishore/Kishore Edu/Studies/sem 4/DAA/week 6$ gcc QuickSort.c -o on
e
kishore@kishore-LOQ-15IRX9:/media/kishore/Kishore Edu/Studies/sem 4/DAA/week 6$ ./one
CH.SC.U4CSE24222
Enter number of elements: 5
Enter 5 elements: 56
64
76
54
34
Choose Pivot Element:
1. First Element
2. Last Element
3. Random Element
Choice: 1
Sorted array: 34 54 56 64 76
kishore@kishore-LOQ-15IRX9:/media/kishore/Kishore Edu/Studies/sem 4/DAA/week 6$
```

kishore@kishore-LOQ-15IRX9: /media/kishore/Kishore Edu/Studies/sem 4/DAA/week 6

```
kishore@kishore-LOQ-15IRX9:/media/kishore/Kishore Edu/Studies/sem 4/DAA/week 6$ gcc QuickSort.c -o on
e
kishore@kishore-LOQ-15IRX9:/media/kishore/Kishore Edu/Studies/sem 4/DAA/week 6$ ./one
CH.SC.U4CSE24222
Enter number of elements: 4
Enter 4 elements: 56
76
23
99
Choose Pivot Element:
1. First Element
2. Last Element
3. Random Element
Choice: 3
Random Pivot Element Chosen: 99
Random Pivot Element Chosen: 76
Random Pivot Element Chosen: 23
Sorted array: 23 56 76 99
kishore@kishore-LOQ-15IRX9:/media/kishore/Kishore Edu/Studies/sem 4/DAA/week 6$ 
```

2. Write a c program for Quick Select

Code:

```
#include <stdio.h>

// 1. Helper function to swap two elements
void swap(int* a, int* b) {
    int t = *a;
    *a = *b;
    *b = t;
}

// 2. Partition function (Required for Quick Select)
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

// 3. Quick Select function
int quickSelect(int arr[], int low, int high, int k) {
    // If k is smaller than number of elements in array
    if (k > 0 && k <= high - low + 1) {
```

```
        // Now the compiler knows where to find partition()
        int index = partition(arr, low, high);

        // If index is same as k, we found the element
        if (index - low == k - 1)
            return arr[index];

        // If index is more, look in the left subarray
        if (index - low > k - 1)
            return quickSelect(arr, low, index - 1, k);

        // Else look in the right subarray
        return quickSelect(arr, index + 1, high, k - index + low - 1);
    }
    return -1;
}

int main() {
    // Printing your roll number
    printf("Roll No: CH.SC.U4CSE24222\n");
    printf("Algorithm: Quick Select\n-------------------------\n");

    int arr[] = {10, 4, 5, 8, 6, 11, 26};
    int n = sizeof(arr) / sizeof(arr[0]);
    int k = 3; // We want to find the 3rd smallest element

    int result = quickSelect(arr, 0, n - 1, k);

    if (result != -1) {
```

```
        printf("The %drd smallest element is: %d\n", k, result);
    } else {
        printf("Invalid value for k\n");
    }

    return 0;
}
```

OUTPUT: